

Правила подключения клиентов к сервису STOMP, когда не используются платформенные библиотеки



Функционал устарел

Для взаимодействия с функционалом STOMP используйте платформенные клиенты.

Для взаимодействия клиентов (веб-браузеры и СБИС-плагины) с сервисом STOMP (по названию протокола взаимодействия) используется транспорт на основе библиотеки [SockJS](#) и протокол сообщений STOMP.

Точкой для подключения клиентов к сервису STOMP является stomp, а также сайт указывают с префиксом stomp.

Пример:

```
http://stomp-online.sbis.ru/stomp/
```

При работе с данным сервисом существует несколько основных правил.

Правило № 1. Получение информации о пользователе

При работе с сервисом возникает необходимость указывать персональные ресурсы пользователя. Для безопасности все идентификаторы пользователя преобразуются в некий хэш, который далее используется в именах персональных ресурсов. Чтобы получить такие идентификаторы, нужно обратиться к специальному ресурсу `/!hash/` на диспетчерах сайтов.

Пример:

```
https://online.sbis.ru/!hash/
```

Естественно при этом нужно иметь валидную сессию пользователя.

В результате запроса вернётся JSON-объект вида:

```
1  {
2      "result": {
3          "domain": "stomp-online.sbis.ru",
4          "stomp_ttl": 300,
5          "user": "e73edb68ad23c1166afdafdc10b5ac0a",
6          "client": "06fd0e44ce220d5786fcabebe99c1465",
7          "sid": "00000003-00a06a67-00bd-d1cb6969144a453e",
8          "global": "",
9          "exchange": "online.sbis.ru",
10         "stomp_maxmsgs": 300,
11         "path": "\/stomp\/",
12         "cid": "00000003",
13         "uid": "00a06a67"
14     }
15 }
```

Правило № 2. Первый запрос при подключении к сервису STOMP производится на /info

Согласно поведению библиотеки SockJS, первый запрос всегда должен быть на /info. Пример:

```
http://stomp-online.sbis.ru/stomp/info
```

В ответе возвращается информация о возможностях сервера в виде JSON-объекта. Например:

```
1 {  
2   "websocket": true  
3 }
```

В случае сервиса STOMP этот запрос получает специальное приложение (менеджер web-узла), которое подготавливает всё необходимое на сервере для корректной работы пользователя. Таким образом, чтобы клиент смог подключиться без ошибок, он обязан в начале каждой сессии подключения запрашивать данный ресурс.

Используемые протоколы для подключения со стороны клиента

Подключение из веб-браузера

Для подключения из веб-браузера используется собственный легковесный клиент SockJS. В первую очередь, библиотека пытается использовать нативный WebSocket объект, который есть во всех современных веб-браузерах. Но иногда пользователи сидят за прокси-серверами, которые не передают WebSocket. В таких случаях SockJS пытается использовать другую технологию для связи с сервером: xhr-streaming (поточковая передача).

Поэтому SockJS настроена на использование протоколов в следующей последовательности:

1. websocket. Использование нативного объекта WebSocket в веб-браузере.
2. xhr-streaming. Использование кросс-доменного XHR стриминга.

Если подключение было закрыто во время работы, то процесс его переустановки начинается с самого начала.

Правило № 3. Подключение и авторизация перед выполнением операций

Для выполнения каких-либо операций сначала необходимо подключиться и авторизоваться. Пример фрейма по протоколу STOMP:

```
1 CONNECT  
2 accept-version:1.1,1.0  
3 heart-beat:60000,0  
4 host:/  
5 login:stomp_user  
6 passcode:stomp_user  
7  
8 ^@
```

Таблица 1. Список используемых заголовков для подключения и авторизации.

Заголовок	Описание
heart-beat	Это два таймера, через которые будут посылать специальные сообщения, указывающие на то, что участник обмена всё ещё "живой". Первый задаёт интервал отправки со стороны клиента, второй – со стороны сервера. В большинстве случаев состояние участника можно понять по состоянию нижележащего

	подключения, но так как клиент может работать через polling или streaming (у которых нет единого подключения), то необходимо включить таймер с его стороны, чтобы сервер корректно понимал, что клиент ещё "на связи".
login и passcode	Позволяют авторизовать нас как валидного пользователя. Со стороны клиентов необходимо подключаться, используя аккаунт stomp_user/stomp_user.

Если всё прошло успешно, то сервер вернёт CONNECTED.

Правило № 4. Подписка на персональный обменник

Пример фрейма по протоколу STOMP:

```

1 SUBSCRIBE
2 receipt:692f0237e0f4fdc92c3258bf8bfe7714
3 id:7dfe7531-addb-f00c-65c2-793fbf388653-
  692f0237e0f4fdc92c3258bf8bfe7714
4 persistent:true
5 auto-delete:false
6 prefetch-count:10
7 ack:client
8 destination:/exchange/<sitename>:<user_hash>
```

Таблица 2. Список используемых заголовков для подписки на персональный обменник.

Заголовок	Описание
receipt	Может быть произвольной строкой. Её значение будет возвращено в ответе на подписку – RECEIPT (чтобы идентифицировать запрос). Если этот заголовок опущен, то при успешной подписке система ничего не ответит. Так как мы делаем подписку на персональный обменник, то можно использовать хэш пользователя, полученный на предыдущем этапе.
id	Рекомендуется выбирать случайным для каждого разного устройства, но сохранять его в рамках одного устройства (так как от него зависит имя очереди). Если два устройства будут подключены с одинаковым id, то они будут получать сообщения "по очереди", что не соответствует задаче. Для веб-браузера это может быть некий хэш, полученный из идентификатора браузера и текущей даты и сохранённый в локальное хранилище для дальнейшего постоянного использования.
persistent	Должен быть true. В таком случае RabbitMQ будет создавать постоянные очереди, которые "переживут" переподключение клиента. А также имя конечной очереди всегда будет одинаковым и составлять функцию от id.
Auto-delete	Данный заголовок отвечает за автоматическое удаление очередей при отключении. Во избежание удаления при кратковременном отключении при разрыве связи устанавливается в значение false.
Prefetch-count	Максимальное количество неподтвержденных сообщений клиентом. При достижении этого значения сервер перестает отправлять клиенту сообщения до следующего подтверждения. Не используется в режиме автоматического подтверждения no_ack.
Ack	Режим подтверждения доставки сообщений.

	client — клиент сам отправляет подтверждение серверу о получении сообщения. no_ack — доставка без подтверждения, сервер считает все отправленные сообщения доставленными.
destination	Указывается имя обменника для подключения. Здесь имя состоит из двух частей: имя сайта (sitename) и хэш пользователя (user_hash).

Правило № 5. Подписка на канализированные события.

Пример фрейма по протоколу STOMP:

```

1 SUBSCRIBE
2 receipt:692f0237e0f4fdc92c3258bf8bfe7714
3 id:7dfe7531-addb-f00c-65c2-793fbf388653-
  692f0237e0f4fdc92c3258bf8bfe7714-<evt_name>-<user/client-id>
4 persistent:true
5 auto-delete:false
6 prefetch-count:10
7 ack:client
8 destination:/exchange/<sitename>.channel/<user/client-id>${evt_name}

```

Таблица 3. Список используемых заголовков для подписки на канализированные события.

Заголовок	Описание
receipt	Может быть произвольной строкой. Её значение будет возвращено в ответе на подписку – RECEIPT (чтобы идентифицировать запрос). Если этот заголовок опущен, то при успешной подписке система ничего не ответит. Так как мы делаем подписку на персональный обменник, то можно использовать хэш пользователя, полученный на предыдущем этапе.
id	Рекомендуется выбирать случайным для каждого разного устройства, но сохранять его в рамках одного устройства (так как от него зависит имя очереди). Если два устройства будут подключены с одинаковым id, то они будут получать сообщения "по очереди", что не соответствует задаче. Для веб-браузера это может быть некий хэш, полученный из идентификатора браузера и текущей даты и сохранённый в локальное хранилище для дальнейшего постоянного использования. При подписке на канализированные события к id также добавляются название события и id клиента/пользователя.
persistent	Должен быть true. В таком случае RabbitMQ будет создавать постоянные очереди, которые "переживут" переподключение клиента. А также имя конечной очереди всегда будет одинаковым и составлять функцию от id.
Auto-delete	Данный заголовок отвечает за автоматическое удаление очередей при отключении. Во избежание удаления при кратковременном отключении при разрыве связи устанавливается в значение false.
Prefetch-count	Максимальное количество неподтвержденных сообщений клиентом. При достижении этого значения сервер перестает отправлять клиенту сообщения до следующего подтверждения. Не используется в режиме автоматического подтверждения no_ack.

Ack	Режим подтверждения доставки сообщений. client — клиент сам отправляет подтверждение серверу о получении сообщения. no_ask — доставка без подтверждения, сервер считает все отправленные сообщения доставленными.
destination	Указывается имя обменника для подключения. Здесь имя состоит из трех частей: имя сайта (sitename).channel, id пользователя либо клиента и названия события. Например: destination:/exchange/test-online.sbis.ru.channel/0057a338\$news.likeadded.